من مجلدات Laravel إلى نظام إدارة المهام

درس تطبيقي على مشروع PMT

نبدأ بالملفات التي سنفتحها أثناء الدرس، ثم نشرح MVC. بعد ذلك نتتبّع مهمة واحدة من حالة الاستخدام إلى الجداول وفئات PHP، ونقرأ كيف يستقبل Laravel طلب إسنادها إلى طالب.

UML ERD MVC FILES

خطة الدرس

02	بنية مشروع Laravel	FILES
03	ما MVC؟ وما فائدته؟	MVC
04	حالات الاستخدام	01
05	استخراج قواعد العمل	02
06	مخطط الكيانات والعلاقات	03
07	من الرسم إلى Eloquent	04
08	مخطط فئات النماذج	05
09	مخطط فئات المتحكّمات	06
10	المسارات ودوال المتحكّم	07
11	التحقق من طلب الإسناد	08
12	تسلسل طلب الإسناد	08
13	تمرين تطبيقي	09

الرسومات والتواقيع مستندة إلى ملفات PMT في C:\PMT\backend.

بنية مشروع Laravel

أين نجد الملفات التي سنستخدمها في الدرس؟

هذه بنية مشروع PMT الحالية، مع توسيع المجلدات التي سنفتحها أثناء الدرس.

المجلد	ماذا نجد فيه؟
`routes/`	المسارات (Routes). نفتح `web.php` لطلبات المتصفح، و`api.php` لطلبات واجهة API.
`app/Http/Controllers/`	المتحكّمات (Controllers). هنا نجد الدالة التي تنفّذ الطلب، مثل عرض مهمة أو إسنادها.
`app/Models/`	النماذج (Models). نفتح `Task.php` لنعرف كيف تصل المهمة إلى تصنيفها وطلابها.
`resources/views/`	واجهات العرض (Views). هنا قوالب Blade لصفحات المهام، ومنها نموذج اختيار الطالب.
`database/migrations/`	ملفات ترحيل قاعدة البيانات (Migrations). تعرّف الجداول والمفاتيح والقيود.
`database/seeders/`	ملفات تعبئة البيانات الأولية (Seeders). يعبّئ `DatabaseSeeder.php` بيانات التدريب.
`tests/Feature/`	اختبارات الميزات (Feature Tests). ترسل طلبات إلى التطبيق وتفحص الاستجابة والبيانات المحفوظة.

PMT · PROJECT TREE

```
C:\PMT\backend
├── app/
│   ├── Http/Controllers/
│   ├── Models/
│   └── Providers/
├── bootstrap/app.php
├── config/
├── database/
│   ├── migrations/
│   └── seeders/
├── docs/
├── public/index.php
├── resources/views/
│   ├── layouts/
│   └── tasks/
│       ├── index.blade.php
│       ├── create.blade.php
│       ├── show.blade.php
│       └── edit.blade.php
├── routes/
│   ├── web.php
│   └── api.php
├── storage/
├── tests/Feature/
└── vendor/
```

ما MVC؟ وما فائدته؟

كيف تتوزع مسؤوليات التطبيق؟

MVC اختصار لـ **النموذج، وواجهة العرض، والمتحكّم** (Model–View–Controller). يوزّع هذا النمط العمل بين الأجزاء الثلاثة حتى نعرف أين نقرأ الكود وأين نجري التعديل.

الجزء	مسؤوليته	مثال من PMT
النموذج (Model)	يمثّل بيانات التطبيق ويعرّف طريقة الوصول إلى السجلات المرتبطة.	`Task` يعرّف علاقتَي `category()` و`students()`.
واجهة العرض (View)	ترتّب البيانات وتعرضها للمستخدم في صفحة.	`tasks/show.blade.php` يعرض تفاصيل المهمة وقائمة الطلاب.
المتحكّم (Controller)	يستقبل الطلب وينسّق قراءة البيانات أو حفظها ثم يختار الاستجابة.	`TaskPageController::show()` يحمّل علاقات المهمة ويمرّر البيانات إلى Blade.

ماذا يحدث عند فتح المهمة رقم 12؟

```
GET /tasks/12  →  Route  →  TaskPageController::show()  →  Task + relationships  →  Blade → HTML
```

يوجّه المسار (Route) الطلب إلى المتحكّم. يحمّل المتحكّم تصنيف المهمة والطلاب المسندة إليهم، ثم يولّد قالب Blade صفحة HTML من البيانات.

> **فائدة MVC** إذا أردنا تغيير شكل قائمة الطلاب نفتح قالب Blade. وإذا أردنا معرفة كيف ترتبط المهمة بالطلاب نفتح نموذج `Task`. ويمكن لمتحكّم الويب ومتحكّم API استخدام العلاقة نفسها، ثم يعيد أحدهما HTML والآخر JSON.

حالات الاستخدام

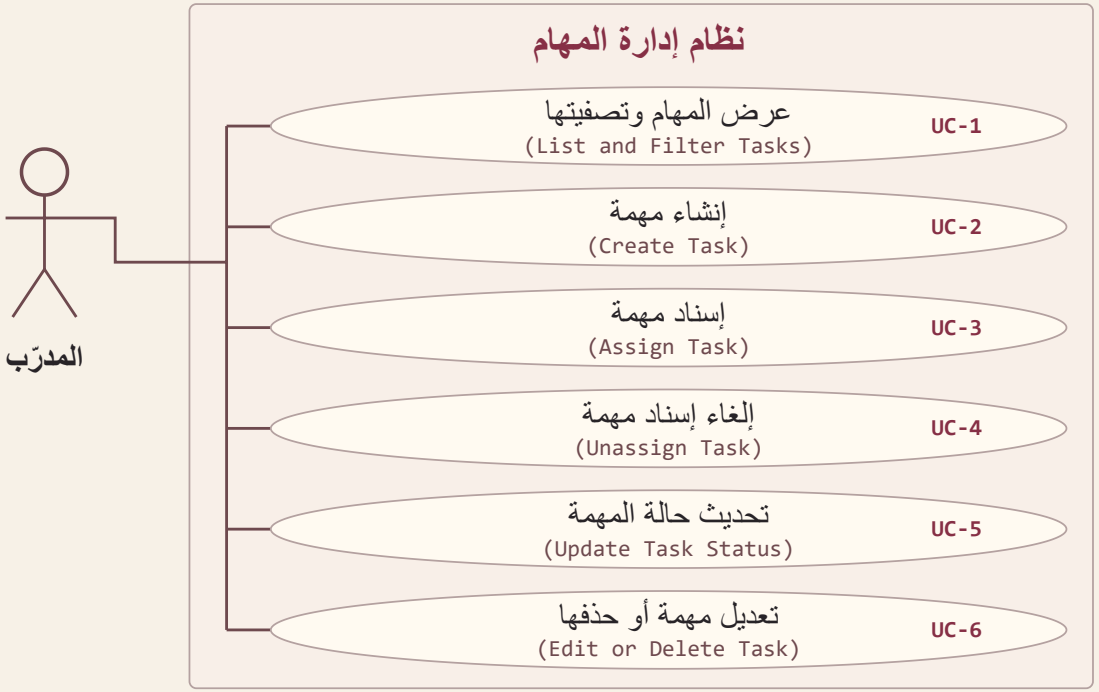
ماذا يريد المدرّب أن ينجز في PMT؟

> أريد إنشاء مهمة «**شرح الربط بين الجداول**» ضمن تصنيف «قواعد البيانات»، ثم إسنادها إلى سارة وعمر وتحديث حالتها عند اكتمالها.

في نظام إدارة المهام (Task Management System)، يختار المدرّب تصنيفًا (Category) للمهمة (Task)، ثم يختار الطالب (Student) الذي يريد إسنادها إليه.

إسناد مهمة (Assign Task)

1. يفتح المدرّب تفاصيل مهمة موجودة ويختار سارة من القائمة.
2. يرسل النموذج. يبحث التطبيق عن سجل الطالب ويحفظ الإسناد ووقته.
3. تظهر سارة في صفحة التفاصيل. يكرر المدرّب الخطوة لإسناد المهمة إلى عمر.



01 مخطط حالات الاستخدام (Use Case Diagram). يظهر المدرّب خارج حدود النظام. ويمثّل كل شكل بيضاوي عملية يستطيع تنفيذها.

افتح في المشروع `routes/web.php` / `resources/views/tasks/show.blade.php`

استخراج قواعد العمل

ما الذي نحفظه؟ وإلى أي سجل تنتمي المعلومة؟

نقرأ عبارات السيناريو ونحدد البيانات التي يجب حفظها. ثم نسأل إن كانت المعلومة تخص سجلًا واحدًا أم الارتباط بين سجلين.

ما ورد في السيناريو	قاعدة العمل (Business Rule)	القرار في التصميم
«أنشئ مهمة ضمن تصنيف»	لكل مهمة تصنيف واحد، وقد يحتوي التصنيف على عدة مهام.	نضع معرّف التصنيف `category_id` في جدول `tasks`.
«أسند المهمة إلى سارة وعمر»	يمكن أن تُسند المهمة إلى عدة طلاب، ويمكن أن يتلقى الطالب عدة مهام.	نستخدم علاقة متعدد إلى متعدد (Many-to-Many) بين `Task` و`Student`.
«سجّل وقت الإسناد»	قد تُسند المهمة إلى سارة اليوم وإلى عمر غدًا.	نحفظ `assigned_at` لكل إسناد على حدة في `student_task`.
«غيّر حالة المهمة إلى مكتملة»	الحالة في PMT تخص المهمة نفسها.	نحفظ `status` في سجل المهمة.
«ألغِ إسناد سارة»	يمكن إزالة ارتباط واحد مع بقاء المهمة وإسناد عمر.	نحذف صف الربط الخاص بسارة وهذه المهمة.

أين نضع الحقل؟ موعد التسليم (Due Date) يخص المهمة ويُحفظ في `tasks.due_date`. ووقت الإسناد (Assignment Time) يخص ارتباط طالب بمهمة ويُحفظ في `student_task.assigned_at`.

مخطط الكيانات والعلاقات

كيف ترتبط الجداول الأربعة؟

كيف نقرأ الأعداد على الخط؟

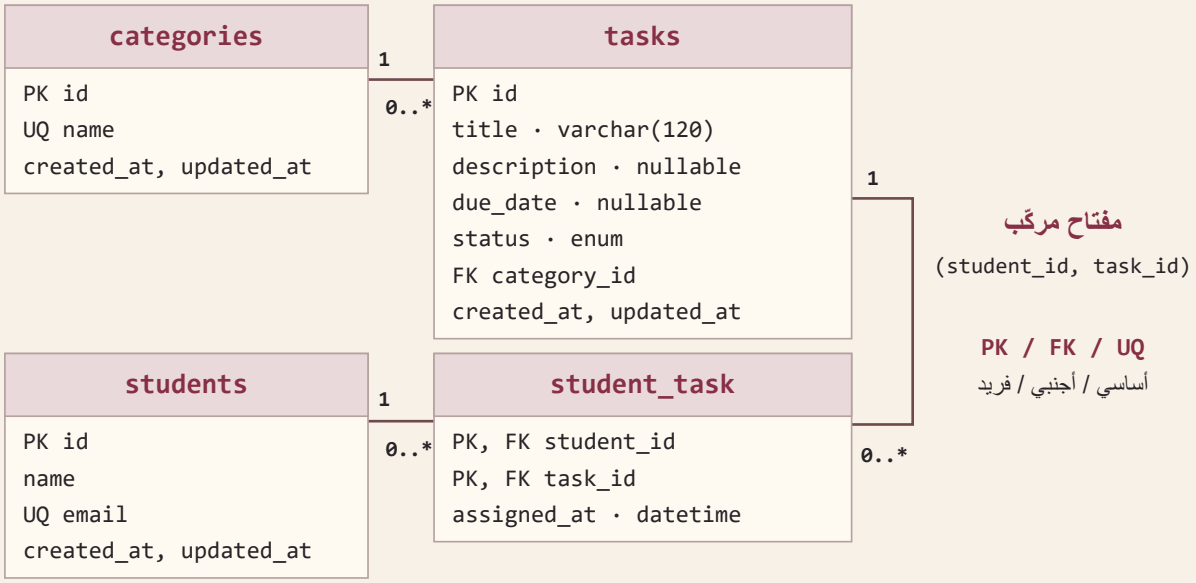
لكل مهمة تصنيف واحد. وقد يضم التصنيف صفرًا أو أكثر من المهام. يحمل `tasks.category_id` معرّف ذلك التصنيف.

لكل صف إسناد طالب واحد ومهمة واحدة. يجمع المفتاح الأساسي المركّب (Composite Primary Key) معرّفَي الطالب والمهمة، بالترتيب `(student_id, task_id)`.

مثال من السيناريو

`(3, 12)` هو إسناد سارة و`(8, 12)` هو إسناد عمر. يسمح المفتاح بالصفّين، ويمنع تكرار الزوج نفسه.

عند حذف طالب أو مهمة، يزيل الحذف المتتابع (Cascade Delete) إسناداتهما. ويمنع قيد الحذف (Restrict on Delete) حذف تصنيف ما دامت له مهام.



02 مخطط الكيانات والعلاقات (Entity–Relationship Diagram). يرمز PK إلى مفتاح أساسي (Primary Key)، وFK إلى مفتاح أجنبي (Foreign Key)، وUQ إلى قيد فريد (Unique).

العنوان حتى 120 حرفًا. الوصف والموعد اختياريان. قيم الحالة `pending` و`in_progress` و`done`، والقيمة الافتراضية في قاعدة البيانات `pending`.

افتح في المشروع `database/migrations/2026_11_04_000001_create_training_tables.php`

من الرسم إلى Eloquent

كيف تظهر العلاقات في ملفات المشروع؟

يعرّف ملف ترحيل قاعدة البيانات (Migration) الجداول والمفاتيح والقيود. وتعرّف نماذج Eloquent كيف يصل كود PHP إلى السجلات المرتبطة.

أ. التصنيف والمهام

Migration · tasks

```
$table->foreignId('category_id')
    ->constrained()->restrictOnDelete();
```

app/Models · Task.php + Category.php

```
// Task::category()
return $this->belongsTo(Category::class);

// Category::tasks()
return $this->hasMany(Task::class);
```

تجلب `belongsTo()` تصنيف المهمة، وتجلب `hasMany()` مهام التصنيف. يستخدم الاتجاهان المفتاح نفسه `tasks.category_id`.

ب. الطلاب والمهام

Migration · student_task

```
$table->foreignId('student_id')->constrained()->cascadeOnDelete();
$table->foreignId('task_id')->constrained()->cascadeOnDelete();
$table->dateTime('assigned_at');
$table->primary(['student_id', 'task_id']);
```

app/Models · Task.php + Student.php

```
// Task::students()
return $this->belongsToMany(Student::class)
    ->withPivot('assigned_at');

// Student::tasks()
return $this->belongsToMany(Task::class)
    ->withPivot('assigned_at');
```

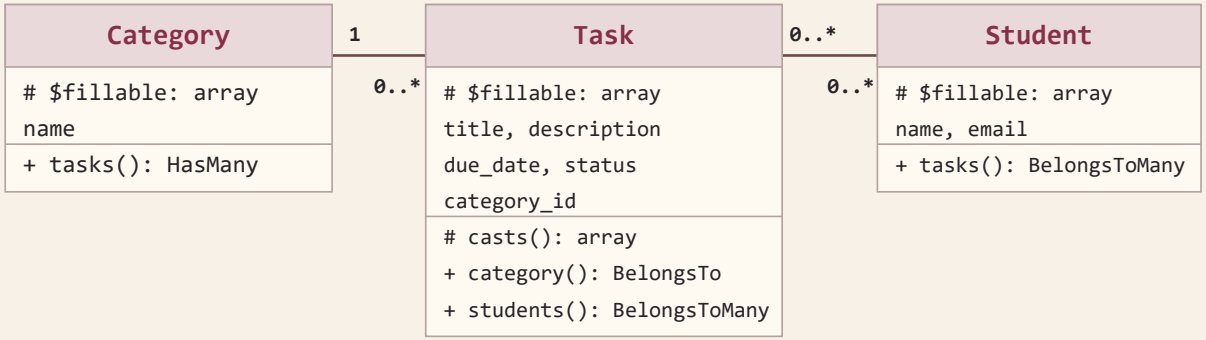
تطلب `withPivot()` تحميل وقت الإسناد من جدول الربط (Pivot Table). نقرأه من `$student->pivot->assigned_at` عند جلب طلاب المهمة.

مخطط فئات النماذج

كيف نقرأ تمثيل الفئة في مخطط UML؟

كيف نقرأ تمثيل الفئة؟

1. الجزء العلوي يعرض اسم الفئة (Class Name).
2. الجزء الأوسط يعرض الخصائص (Properties). تحدّد `$fillable` الحقول التي يقبل النموذج تعبئتها دفعة واحدة (Mass Assignment)، مثل الحقول التي تُمرّر إلى `Task::create($data)`.
3. الجزء السفلي يعرض الدوال (Methods). الرمز `+` يعني عامًا (public)، والرمز `#` يعني محميًا (protected). تتعامل `casts()` مع `due_date` بوصفه تاريخًا.



03 مخطط فئات النماذج (Model Class Diagram). يعرض الخصائص والدوال الفعلية في فئات Eloquent.

افتح في المشروع `app/Models/Category.php` / `app/Models/Student.php` / `app/Models/Task.php`

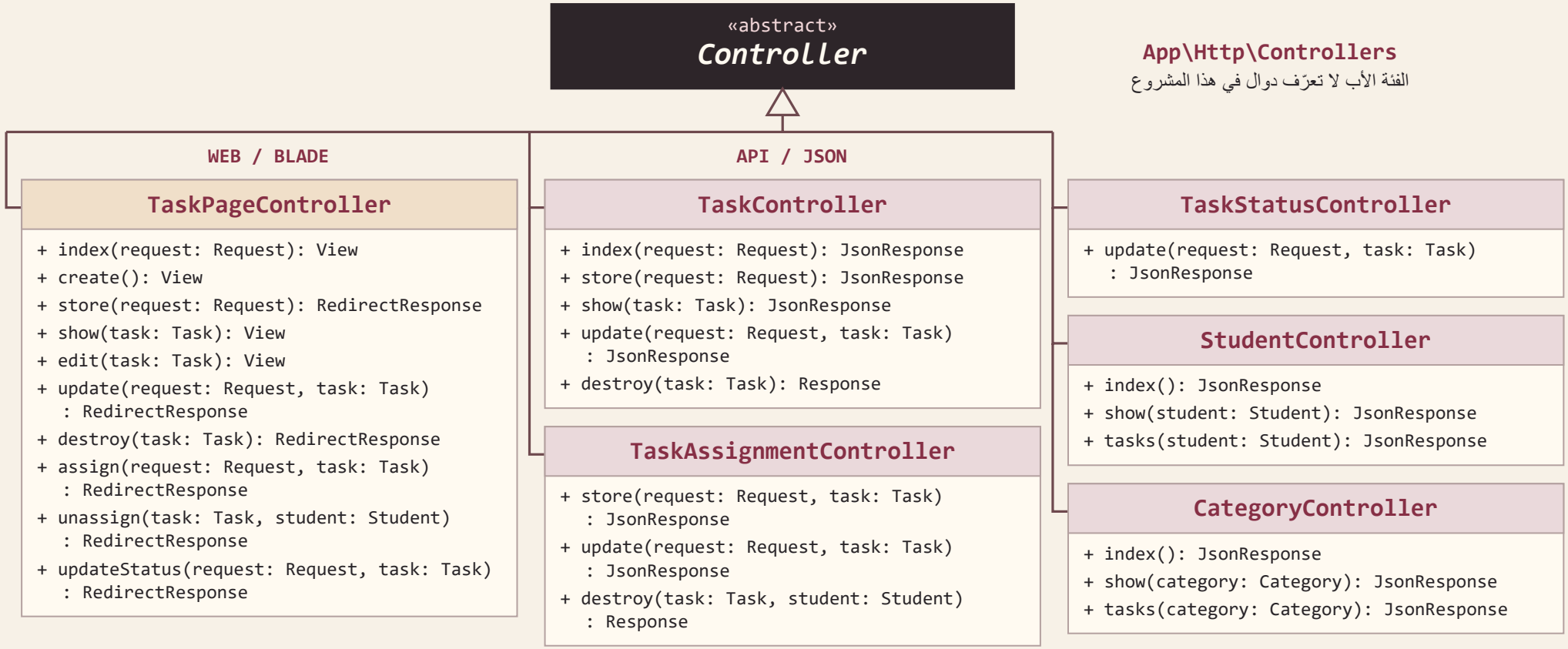
علاقة واحدة في رسمين

يظهر الارتباط بين `Task` و`Student` في مخطط الفئات (Class Diagram)، ويظهر `student_task` في ERD لحفظ صف مستقل لكل إسناد.

من المتحكّم إلى النموذج عند عرض التفاصيل، تستدعي `TaskPageController::show()` السطر `$task->load(['category', 'students']);`. يطابق الاسمان دالتَي العلاقة `category()` و`students()` في `Task`.

مخطط فئات المتحكّمات

الفئة الأب وست فئات مشتقة و25 دالة



04 مخطط فئات المتحكّمات (Controller Class Diagram). يشير المثلث الفارغ إلى الفئة الأب، وهي الفئة المجرّدة Controller (Abstract Class). يقابل الخط extends Controller في PHP.

يظهر توقيع الدالة (Method Signature) في PHP هكذا `public function show(Task $task): View`، وفي UML هكذا `+ show(task: Task): View`. تشير + إلى دالة عامة، و`task: Task` إلى اسم المعامل (Parameter) ونوعه، و`View` إلى نوع الإرجاع (Return Type). ويشير `Response` إلى `\Illuminate\Http\Response`.

افتح في المشروع `app/Http/Controllers/*.php`

المسارات ودوال المتحكّم

أي دالة تستقبل كل طلب؟

نبدأ بطلب (Request)، ونبحث عن المسار (Route) في `routes/web.php` أو `routes/api.php`، ثم نفتح دالة المتحكّم (Controller Action) المشار إليها.

الطلب (Request)	دالة المتحكّم (Controller Action)	الاستجابة أو الأثر
`GET /tasks` `GET /api/tasks`	`TaskPageController::index()` `TaskController::index()`	قائمة المهام. الأول يعرض صفحة Blade، والثاني يعيد JSON.
`GET /tasks/{task}`	`TaskPageController::show()`	صفحة تفاصيل المهمة وتصنيفها والطلاب المسندة إليهم.
`POST /tasks/{task}/assign`	`TaskPageController::assign()`	يحفظ إسناد الطالب الذي يحدّده `student_id` ثم يعيد التوجيه.
`POST /api/tasks/{task}/students` `PUT /api/tasks/{task}/students`	`TaskAssignmentController::store()` `TaskAssignmentController::update()`	يضيف POST الطلاب الواردين في `student_ids`. ويجعل PUT الإسنادات تطابق القائمة المرسلة.
`PATCH /api/tasks/{task}/status`	`TaskStatusController::update()`	يقبل `done`، ويرفض `completed` لأنها خارج القيم المحددة.
`GET /api/students/{student}/tasks` `GET /api/categories/{category}/tasks`	`StudentController::tasks()` `CategoryController::tasks()`	يقرأ المهام المرتبطة بالطالب أو بالتصنيف عبر علاقة `tasks()`.

علاقتان في UML يبيّن `Task $task` في توقيع `TaskStatusController::update()` اعتماد الدالة (Dependency) على نموذج `Task`. ويبيّن `extends Controller` علاقة الوراثة (Inheritance) مع الفئة الأب.

التحقق من طلب الإسناد

ماذا يرسل النموذج، وما الشرط الذي يفحصه Laravel؟

لنفترض أن رقم المهمة `12` ورقم سارة `3`. يرى المدرّب اسم سارة في القائمة، ويرسل نموذج Blade رقمها في `student_id`. معرّف الطالب (Student ID) هو قيمة `id` في صف سارة بجدول `students`.

رقم المهمة يأتي من الرابط

في `POST /tasks/12/assign`، يبحث Laravel عن مهمة رقمها `12`. تسمّى الآلية ربط النموذج بالمسار (Route Model Binding). إذا كانت المهمة غير موجودة، يعيد `404` قبل استدعاء `assign()`.

رقم الطالب يأتي من النموذج

`TaskPageController::assign()` · Validation

```
$request->validate([
    'student_id' => 'required|exists:students,id',
]);
```

ما وصل إلى Laravel	القاعدة التي تُفحص	النتيجة
`student_id=3`، **ويوجد طالب رقمه 3**	`required` تتحقق من وجود قيمة، و`exists:students,id` تبحث عن سجل مطابق.	يمرّ التحقق. إذا كانت سارة غير مسندة للمهمة، يُضاف صف الإسناد `(3, 12)`.
الحقل `student_id` **غائب أو فارغ**	`required` تشترط إرسال قيمة.	يتوقف الطلب قبل الحفظ وتظهر رسالة خطأ في صفحة الويب.
`student_id=999`، **ولا يوجد طالب بهذا الرقم**	`exists:students,id` تشترط أن يطابق الرقم سجلًا موجودًا.	يتوقف الطلب قبل الحفظ وتظهر رسالة خطأ في صفحة الويب.

كيف تصل قيمة غير موجودة؟ قد يفتح المدرّب القائمة قبل حذف سجل طالب، ثم يختاره من الصفحة التي بقيت مفتوحة. يفحص Laravel جدول `students` وقت وصول الطلب.

تسلسل طلب الإسناد

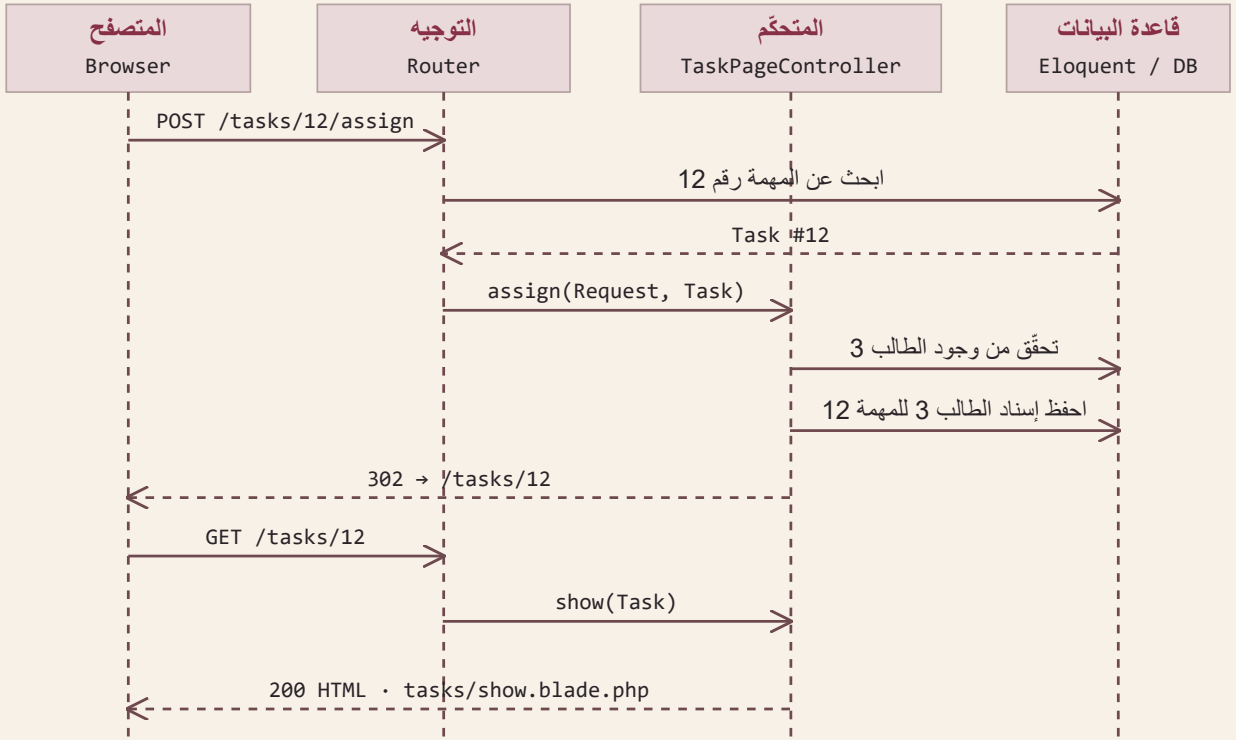
من إرسال النموذج إلى عرض النتيجة

من التحقق إلى عرض النتيجة

1. بعد اجتياز التحقق، تستدعي `assign()` علاقة `$task->students()` ثم `syncWithoutDetaching()` مع `assigned_at`. إذا كانت سارة غير مسندة بعد، يُضاف الصف `(3, 12)` وتبقى بقية الإسنادات.
2. يعيد المتحكّم توجيه المتصفح (Redirect) إلى `/tasks/12`. يرسل المتصفح طلب `GET` جديدًا، وتحمّل `show()` التصنيف والطلاب ثم تعرض صفحة التفاصيل.

الإضافة والاستبدال في API

إذا كانت المهمة مسندة إلى سارة، فإن إرسال `{"student_ids":[8]}` عبر `POST` يضيف عمر ويُبقي سارة باستخدام `syncWithoutDetaching()`. ومع `PUT` تستخدم الدالة `sync()`، فيصبح عمر الطالب الوحيد المسند إلى المهمة.



05 مخطط التسلسل (Sequence Diagram) لمسار النجاح عند إسناد سارة. يحمل الطلب الأول student_id=3 في بيانات النموذج.

افتح في المشروع routes/web.php / app/Http/Controllers/TaskPageController.php

تمرين تطبيقي

توقّع النتيجة قبل قراءة الإجابة

المهمة رقم 12 قيد الانتظار (pending) ولا توجد لها إسنادات. رقم سارة 3 ورقم عمر 8.

1. أُسندت المهمة إلى الطالبين. ما الصفوف التي أُضيفت؟ وأين نقرأ وقت كل إسناد؟
2. تغيّرت الحالة إلى مكتملة (done). أي سجل يتغيّر؟
3. أُلغي إسناد سارة. أي دالة تستقبل طلب الويب؟ وأي صف يُحذف؟
4. وصل طلب إسناد إلى مهمة موجودة ومعه student_id=999، ولا يوجد طالب بهذا الرقم. أين يتوقف الطلب؟

سؤال للتوسّع

إذا أردنا تسجيل تقدّم مستقل لكل طالب في المهمة نفسها، فأين نضع حقل التقدّم؟ برّر اختيارك من خلال السجل الذي يجمع الطالب والمهمة.

الإجابة النموذجية

الإجراء	الأثر
الإسناد	يُضاف (3, 12) و(8, 12) إلى student_task. كل زوج يقرأ معرّف الطالب ثم معرّف المهمة، ويحمل كل صف assigned_at.
تحديث الحالة	يتغيّر tasks.status للمهمة رقم 12 إلى done. تبقى صفوف الإسناد مرتبطة بالمهمة.
إلغاء إسناد سارة	TaskPageController::unassign() تستدعي detach()، فيُحذف (3, 12) وتبقى المهمة وسارة وإسناد عمر.
الطالب 999	يجد Laravel المهمة ويستدعي assign(). تفشل قاعدة exists:students,id قبل كتابة أي صف إسناد.

مرجع بناء دراسة الحالة

Visual Paradigm · From Use Case to MVC Framework